

Sprawozdanie z Laboratorium 5

Przedmiot: Integracja Systemów IT

Temat: Automatyzacja procesów wytwórczych z wykorzystaniem narzędzi CI/CD (GitHub Actions, platforma PaaS Render)

Autor: Anna Godlewska

Wstęp

Celem prac laboratoryjnych było zaprojektowanie i wdrożenie zautomatyzowanego potoku CI/CD (Continuous Integration / Continuous Deployment) dla aplikacji opartej na frameworku Django. Wykorzystano platformę GitHub Actions do automatyzacji testów i weryfikacji kodu oraz środowisko Render.com jako docelową platformę chmurową (PaaS).

Zadanie 1: Konfiguracja i uruchomienie testów jednostkowych

Pierwszym etapem było zagwarantowanie poprawności działania kluczowych komponentów aplikacji w środowisku lokalnym. Uruchomiono wbudowane testy jednostkowe Django w celu sprawdzenia poprawności modeli bazy danych. W trakcie prac konieczne było dostosowanie konfiguracji środowiskowej do poprawnego nawiązania połączenia z bazą danych oraz modyfikacja ustawień PostgreSQL celem rozwiązania błędu odmowy połączenia z hostem lokalnym.

```
C:\Windows\System32\cmd.e X + v - □ X

(venv) C:\Users\mateu\Documents\IntergacjaLaby>docker ps
CONTAINER ID   IMAGE                      COMMAND                  CREATED
STATUS        PORTS                  NAMES
e461aa30832f   intergacjalaby-web        "python manage.py ru..." About an hour ago
Up About an hour 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp intergacjalaby-w
eb-1
4e2211238426   postgres:15              "docker-entrypoint.s..." About an hour ago
Up About an hour 5432/tcp                intergacjalaby-d
b-1

(venv) C:\Users\mateu\Documents\IntergacjaLaby>docker-compose exec web python man
age.py test
Found 2 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
..
-----
Ran 2 tests in 0.455s

OK
Destroying test database for alias 'default'...

(venv) C:\Users\mateu\Documents\IntergacjaLaby>
```

Zadanie 2: Budowa potoku Continuous Integration (CI)

Kolejnym krokiem była automatyzacja procesu testowania po każdej zmianie w repozytorium zdalnym. W tym celu zdefiniowano przepływ pracy w pliku konfiguracji [.github/workflows/main.yml](#).

Proces CI zrealizowano poprzez następujące kroki:

1. Inicjalizacja środowiska bazowego (Ubuntu).
2. Konfiguracja środowiska uruchomieniowego Python.
3. Instalacja pakietów zdefiniowanych w pliku [requirements.txt](#).
4. Weryfikacja jakości kodu za pomocą narzędzia analizy statycznej Linter ([flake8](#)).
5. Wykonanie zautomatyzowanych testów jednostkowych.

✓ Fixing issues Django Syntax Check #19: Commit 6e19e1e pushed by goldixp	feature/cicd-setup	📅 Today at 1:07 AM 🕒 15s
✗ Fixing issues Django CI Pipeline #6: Commit 6e19e1e pushed by goldixp	feature/cicd-setup	📅 Today at 1:07 AM 🕒 1m 53s

Zadanie 3: Optymalizacja potoku integracyjnego

W celu minimalizacji czasu trwania potoku CI/CD, zaimplementowano mechanizm pamięci podręcznej (Cache) dla zależności środowiska Python ([pip](#)). Wykorzystanie akcji [actions/cache](#) wyeliminowało konieczność wielokrotnego pobierania tych samych bibliotek sieciowych, co znacząco zoptymalizowało zużycie zasobów i zredukowało czas trwania etapu instalacji zależności.

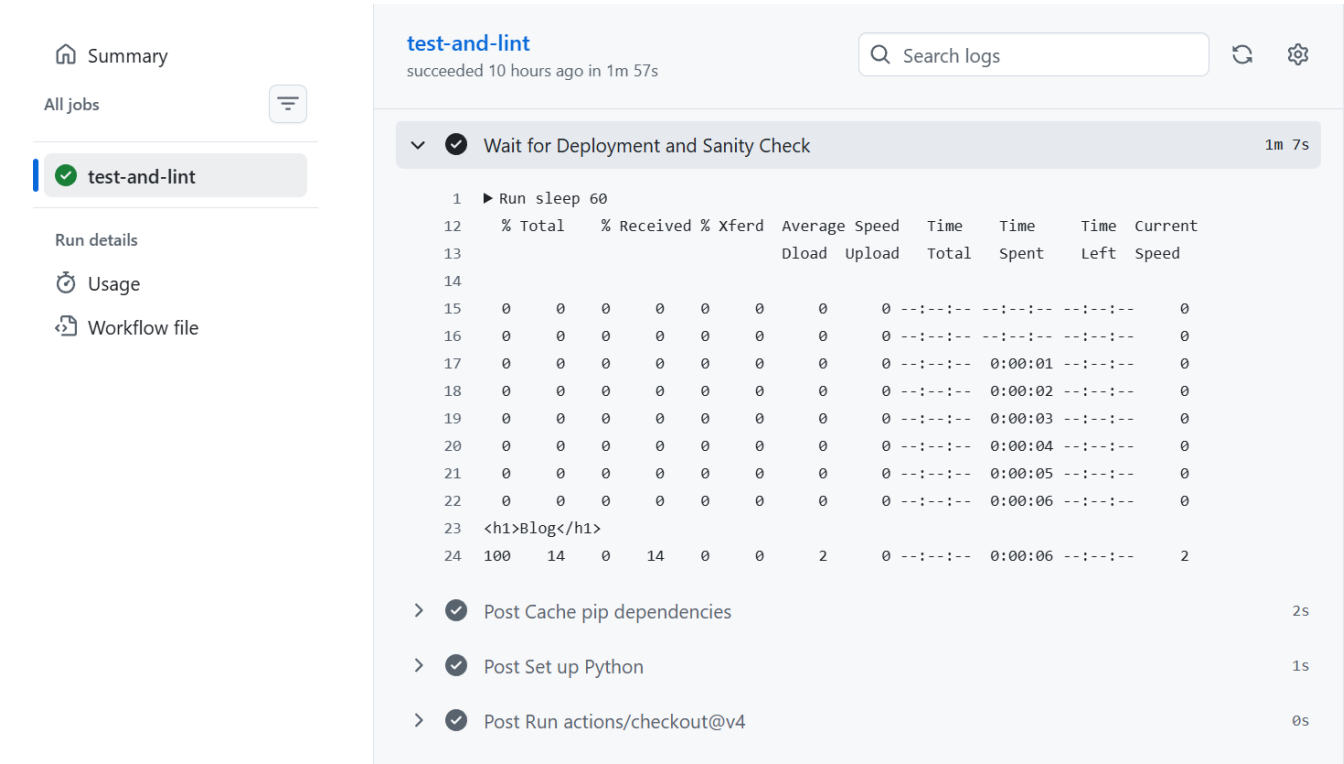
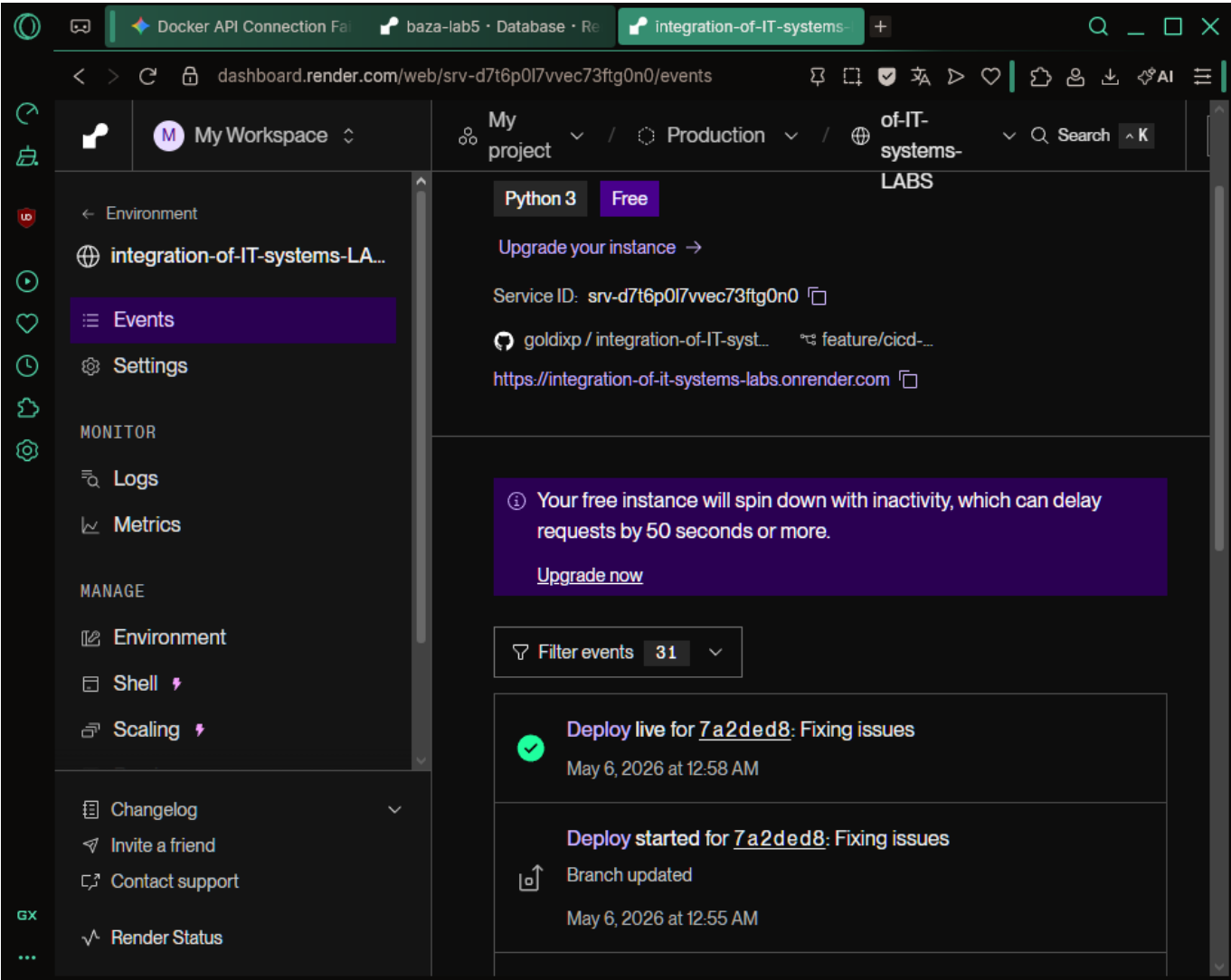
The screenshot displays the GitHub Actions interface for a workflow named 'test-and-lint'. On the left, a sidebar shows the workflow status as 'succeeded 11 hours ago in 47s'. The main panel shows the workflow steps: 'Setup Python', 'Cache pip dependencies' (3s), 'Install Dependencies' (10s), 'Run Linter (flake8)' (1s), 'Run Tests' (2s), and 'Post Cache pip dependencies' (0s). The 'Cache pip dependencies' step is expanded, showing a log where the cache was hit and restored successfully. The log text includes: 'Cache hit for: Linux-pip-a872166b657f50d49d200793526d642e19b14d7c5428270cd3df573cfd84c0cc', 'Received 25165824 of 56996726 (44.2%), 24.0 MBs/sec', 'Received 56996726 of 56996726 (100.0%), 43.5 MBs/sec', 'Cache Size: ~54 MB (56996726 B)', and 'Cache restored from key: Linux-pip-a872166b657f50d49d200793526d642e19b14d7c5428270cd3df573cfd84c0cc'.

Zadanie 4: Ciągłe Wdrożenie (CD) i weryfikacja środowiskowa (Sanity Check)

Zautomatyzowano wdrożenie (Deployment) aplikacji na platformę chmurową Render. Integracja wymagała diagnostyki i rozwiązania napotkanych problemów architektonicznych:

- **Błąd wdrożenia platformy uruchomieniowej:** Wyeliminowano problem z uruchomieniem procesów poprzez deklarację i instalację serwera [unicorn](#) w pliku wymagań.
- **Odrzucenie zapytań HTTP (DisallowedHost):** Zapewniono stabilną komunikację sieciową, poprawnie definiując adres hosta docelowego w zmiennej konfiguracyjnej [ALLOWED_HOSTS](#).

W ramach potoku zaimplementowano również zautomatyzowany test środowiskowy (Sanity Check). Wykorzystano polecenie [curl](#) do weryfikacji dostępności usługi po stronie platformy docelowej, optymalizując czas oczekiwania w celu synchronizacji operacji wdrożeniowych.



Zadanie 5: Integracja wizualnych wskaźników statusu

Ostatnim elementem prac była aktualizacja dokumentacji technicznej repozytorium. Na początku pliku `README.md` umieszczono dynamiczny wskaźnik statusu operacji (Status Badge). Gwarantuje to stały wgląd w aktualną kondycję gałęzi głównej potoku CI/CD, spełniając przy tym standardy utrzymania projektów informatycznych.

 README 



Laboratorium 1: Git, GitHub i przygotowanie środowiska Django

SYMULACJA KONFLIKTU

Celem tego laboratorium jest **praktyczne przygotowanie środowiska programistycznego**, opanowanie systemu kontroli wersji Git oraz integracja z platformą GitHub wraz z automatyzacją.

W ramach realizacji zadań i wymagań zaliczeniowych wykonano następujące kroki:

- Skonfigurowano bezpieczne połączenie z GitHubem przy użyciu kluczy SSH.
- Zainicjalizowano projekt i poprawnie skonfigurowano plik `.gitignore`.
- Przećwiczono pracę z repozytorium na wielu gałęziach (dwie dodatkowe gałęzie oprócz głównej).
- Wykorzystano formatowanie Markdown do stworzenia czytelnej dokumentacji - `python manage.py runserver`.
- Skonfigurowano narzędzie GitHub Actions do automatycznej weryfikacji kodu (testy workflow).

Link do [oficjalnej dokumentacji Django](#).

Podsumowanie

Zaimplementowany system skutecznie łączy środowisko programistyczne z operacyjnym, wdrażając procedury CI/CD w ramach cyklu życia aplikacji. Usunięcie niedociągnięć w konfiguracjach środowiskowych potwierdziło znaczenie dbałości o detale w złożonych architekturach systemów informatycznych.